الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

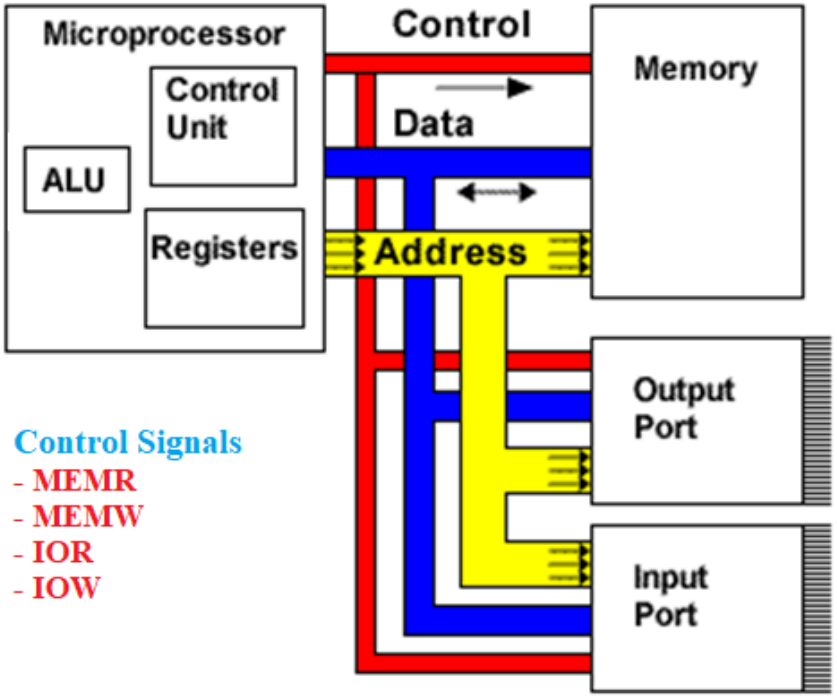
Arab Academy for Science, Technology & Maritime Transport

Lecture 2

Processor Architecture: BIU and EU

Dr. Hesham El Zouka

Processor Schematic Architecture



Address, Data, and Control Buses

- Address Bus: Specifies the memory location that the processor reads data from or writes to.
- Data Bus: Carries the data that has been read from or will be written to the memory location.
- Control Bus: Manages the flow of information between components, indicating whether an operation is a read or write and ensuring it occurs at the correct time.

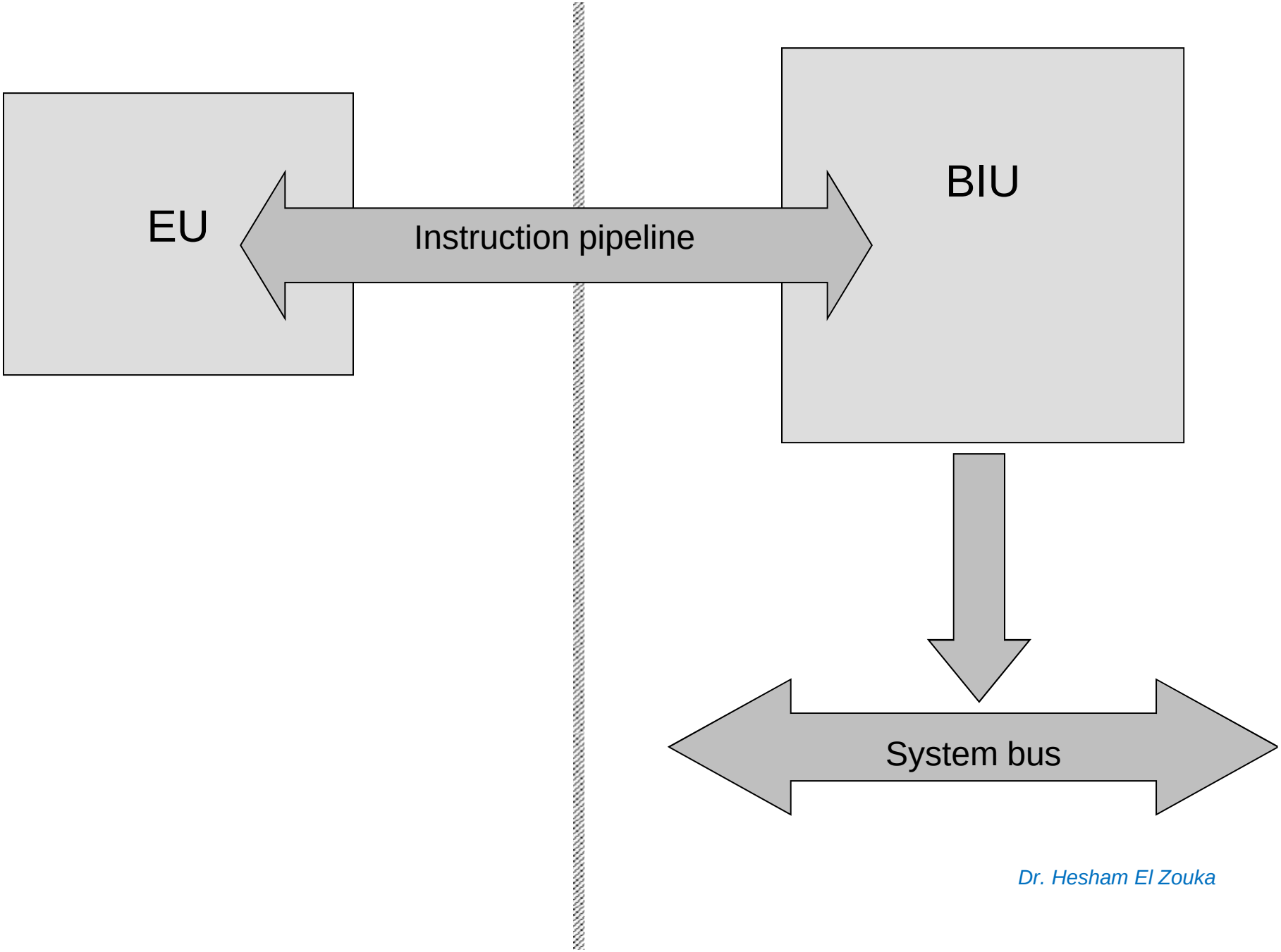
Internal architecture of the 88/86

- Both employ parallel processing

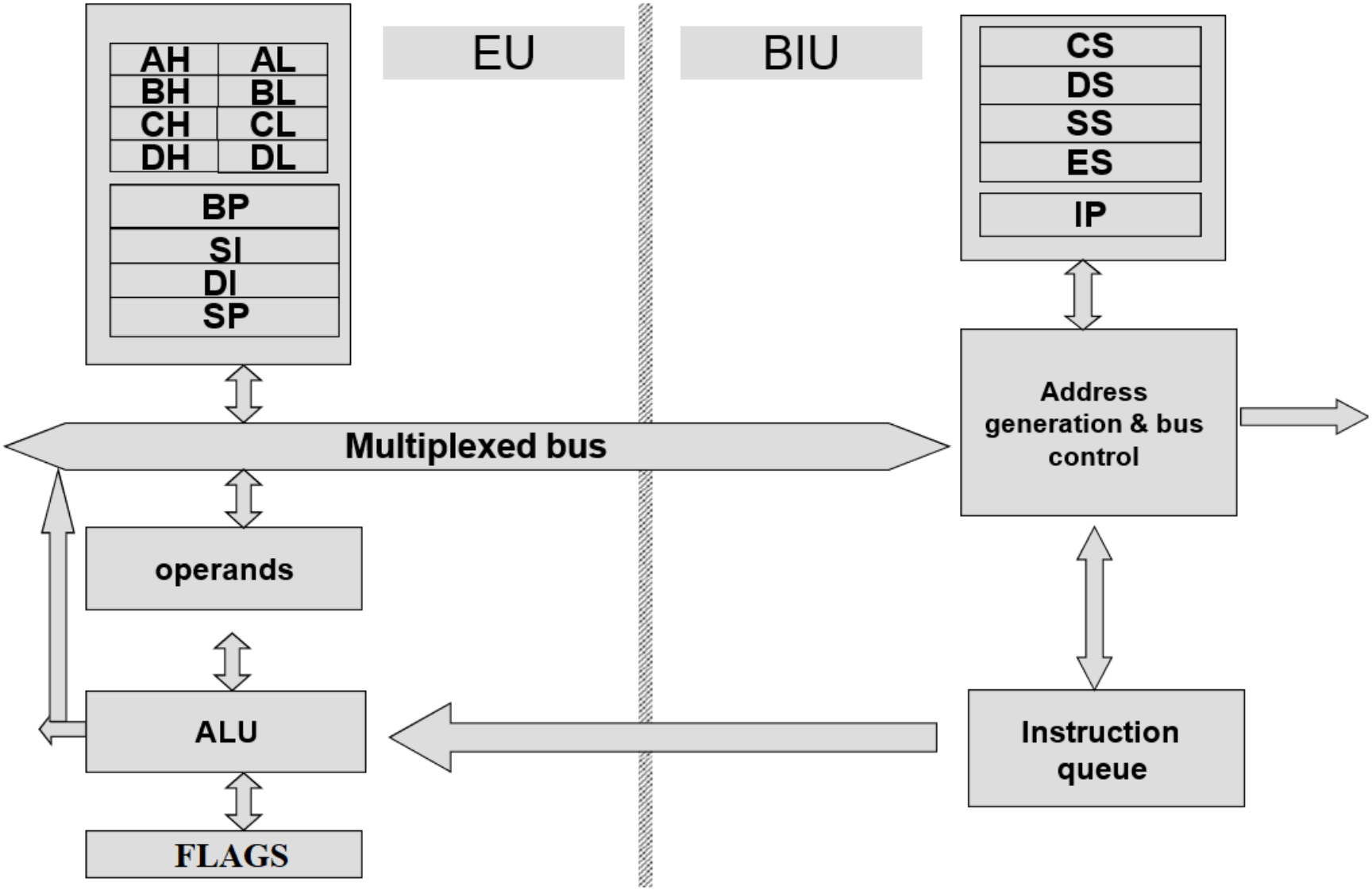
- The processor architecture utilizes **parallel processing** by employing two processing units:
  - 1. BIU: Bus Interface Unit
  - 2. EU: Execution Unit

- Both units work simultaneously enabling instruction fetch and execution to occur independently. This parallelism allows for faster and more efficient operation.

Internal architecture of the 88/86



Internal architecture of the 88/86



Key Components of the BIU

The Bus Interface Unit is responsible for managing external communications. Key functions include:

- **Instruction fetching**
- **Memory read/write operations**
- **I/O read/write operations**
- **Instruction queuing**

Components within the BIU:

- **Segment Registers**: (CS, DS, SS, ES)
- **Instruction Pointer (IP)**
- **Address Generation Adder**
- **Bus Control Logic**
- **6-Byte Instruction Queue (FIFO)**

This architecture implements **pipelining**, where instructions are fetched and queued for execution in the Execution Unit.

	cycle 1	cycle 2	cycle 3	cycle 4	cycle 5	cycle 6
Instruction 1	Fetch	Decode	Execute			
Instruction 2		Fetch	Decode	Execute		
Instruction 3			Fetch	Decode	Execute	
Instruction 4				Fetch	Decode	Execute

Key Components of the Execution Unit

The Execution Unit (EU) handles:

- Decoding and executing instructions
- Accessing data from general-purpose registers
- Checking and updating control flags
- Generating operand addresses when necessary
- Issuing commands to the BIU for memory and I/O operations

Components within the EU:

- ALU (Arithmetic Logic Unit)
- Status and Control Flags
- General-Purpose Registers
- Temporary Operand Registers

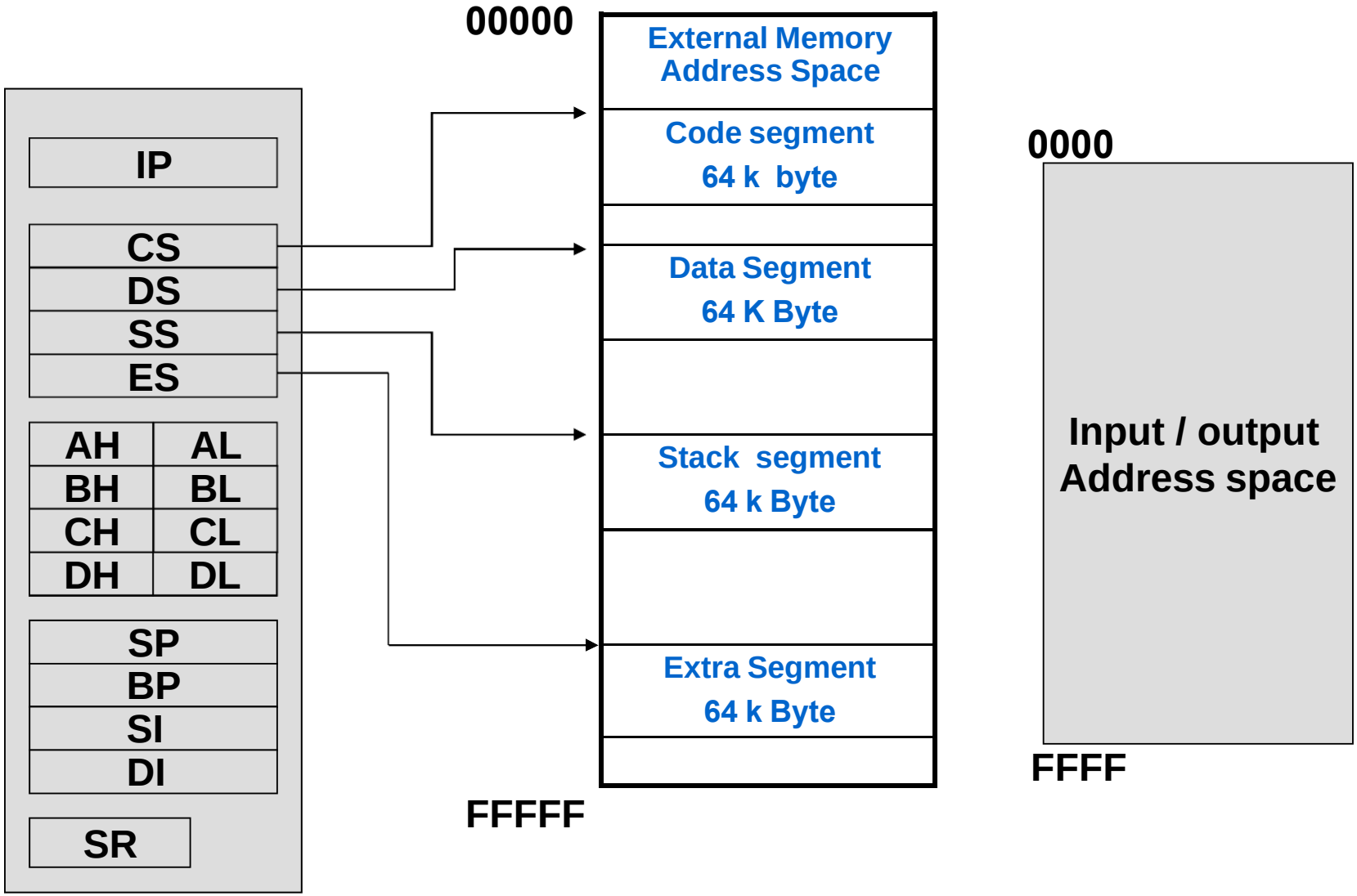
Registers in the 8088/8086

- **The registers can be categorized as follows:**

Category	Bits	Register names
General	16	AX, BX, CX, DX
	8	AH, AL, BH, BL, CH, CL, DH, DL
Pointer	16	SP, BP
Index	16	SI, DI
Segment	16	CS, DS, SS, ES
Instruction	16	IP
Flag	16	FR

Software Model of 8088/8086

- The 8088/8086 processor uses an external memory address space that is divided into segments:



Origin & definition of segments

A segment is an area of memory:

- **Includes up to *64k bytes*.**
- **Each segment is associated with a specific register that stores the *base address*:**
  - **CS: Code Segment**
  - **SS: Stack Segment**
  - **DS: Data Segment**
  - **ES: Extra Segment**
- **Each segment starts at an address divisible by 16, and a maximum of 4 segments can be active at once. The total memory space available for activation is 256 KB (4 × 64 KB).**

To achieve compatibility with the 8085

Introduction to program segments

- An assembly program consists of three (or four) segments:
  - **Code Segment:** Contains the program's instructions that are executed by the processor.
  - **Data Segment:** Stores data such as variables and constants to be processed.
  - **Stack Segment:** Temporarily holds data during function calls and register saves, using a Last-In-First-Out (LIFO) approach.
  - **Extra Segment:** Used for additional data storage, often for string operations or to extend the data segment.

*Only **four of these segments** can be activated at a time.*

Base address for a memory segment

The location of each memory segment is stored in a dedicated **register** within the **Bus Interface Unit (BUI):**

» **CS register** holds **Code Segment** Address

» **SS register** holds **Stack Segment** Address

» **DS register** holds **Data Segment** Address

» **ES register** holds **Extra Segment** Address

How much memory can be activated at a time?

- **Since only four segments can be activated at a time, the total memory that can be accessed at any given time is:**

4 x 64 KB = 256 KB.

Logical & physical address

1. ***Physical Address:*** The 20-bit address placed on the memory bus, defining the actual memory location.(range ?)

– ***Range:*** 00000 to FFFFFH (1 MB).

– ***Example:*** Physical address 12345H points directly to a memory.

2. ***Offset Address:*** a location within a 64 KB segment of memory. (range ?)

– ***Range:*** 0000 to FFFFH (64 KB).

– ***Example:*** Offset 0010H points to the 16th byte in a segment.

3. ***Logical Address:*** *A combination of a segment value & an offset, converted to a physical address.*

   Formula:

   Physical Address = *(Segment Value * 16) + Offset*

   Example: *Logical address 1234: 0020 gives the physical address: 12360H.*

Logical vs. Physical Address in the Code Segment

In the code segment (CS), the CS and IP (Instruction Pointer) registers hold the logical address of the instruction to be executed.

- The format is CS:IP

Where:

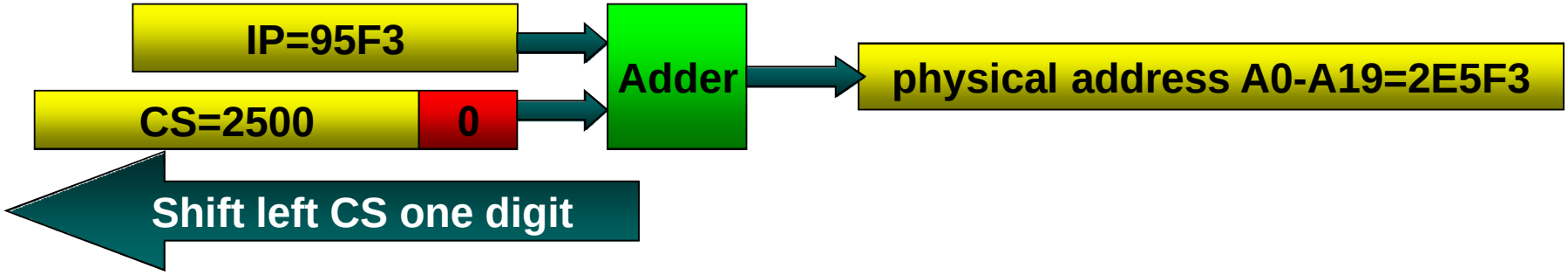
- **CS:** Contains the segment value.
- **IP:** Contains the offset (location within the segment).

The CPU converts this logical address into a physical address using the formula:

Physical Address = **(CS * 16)** + **IP**

Example: If **CS** = **2500H** and **IP** = **95F3H**, the physical address would be:

(2500*H* * 16) + **95F3*H*** = **25000*H*** + **95F3*H*** = **2E5F3*H***



Example 1

If CS =2567 and IP=2341, find:

- **» The logical address (format).**
- **» The off-set address.**
- **» The physical address.**
- **» The lower range in the code segment.**
- **» The upper range in the code segment.**

Example 2

Dynamic Behavior of the Segment and Offset Concept

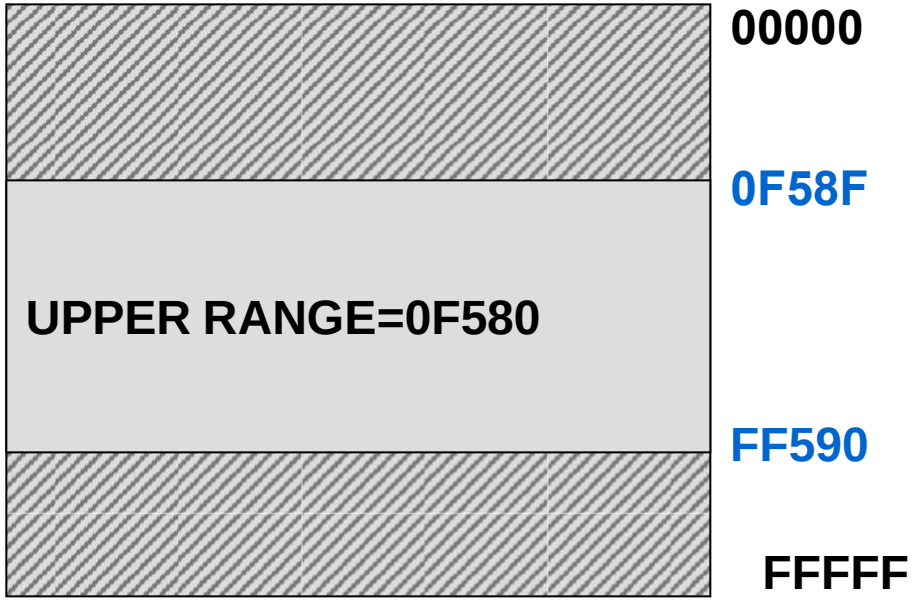
Find the physical address for each of the following logical addresses:

- » **1000:5020**
- » **1500:0020**
- » **1502:0000**
- » **1400:1020**
- » **1302:2000**

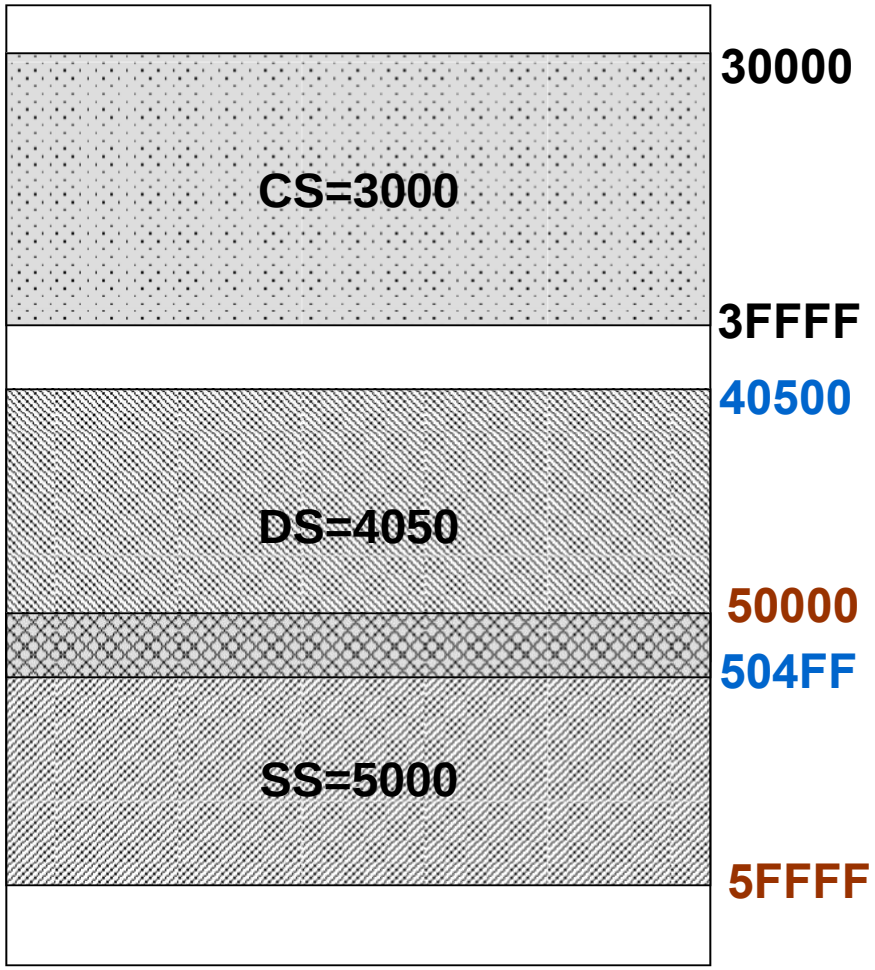
Example 3

Wrap-Around

What is the rang of the physical addresses if CS=FF59 ?



Segment Overlapping



Instruction Pointer (IP)

- **16-Bit Register:** The IP register is 16 bits in size.
- **Contains Offset**: It holds the offset of the next instruction to be fetched from the current Code Segment, rather than the actual physical address.
- **Incrementing Value:** Every time an instruction is fetched, the 8088/8086 microprocessor updates the value of IP by incrementing it by 2 (for a typical instruction size).

Stack segment

- **What is a stack?**
  - » A segment of RAM used by the CPU for temporary storage of data, such as function parameters, return address, and local variables.

- **Why is it needed?**
  - » The number of registers inside the CPU is insufficient for all required data storage, necessitating the use of a stack.

- **How Are Stacks Accessed?**
  - » The SS & SP registers are the two main registers used to access the stack.

Pushing onto the Stack

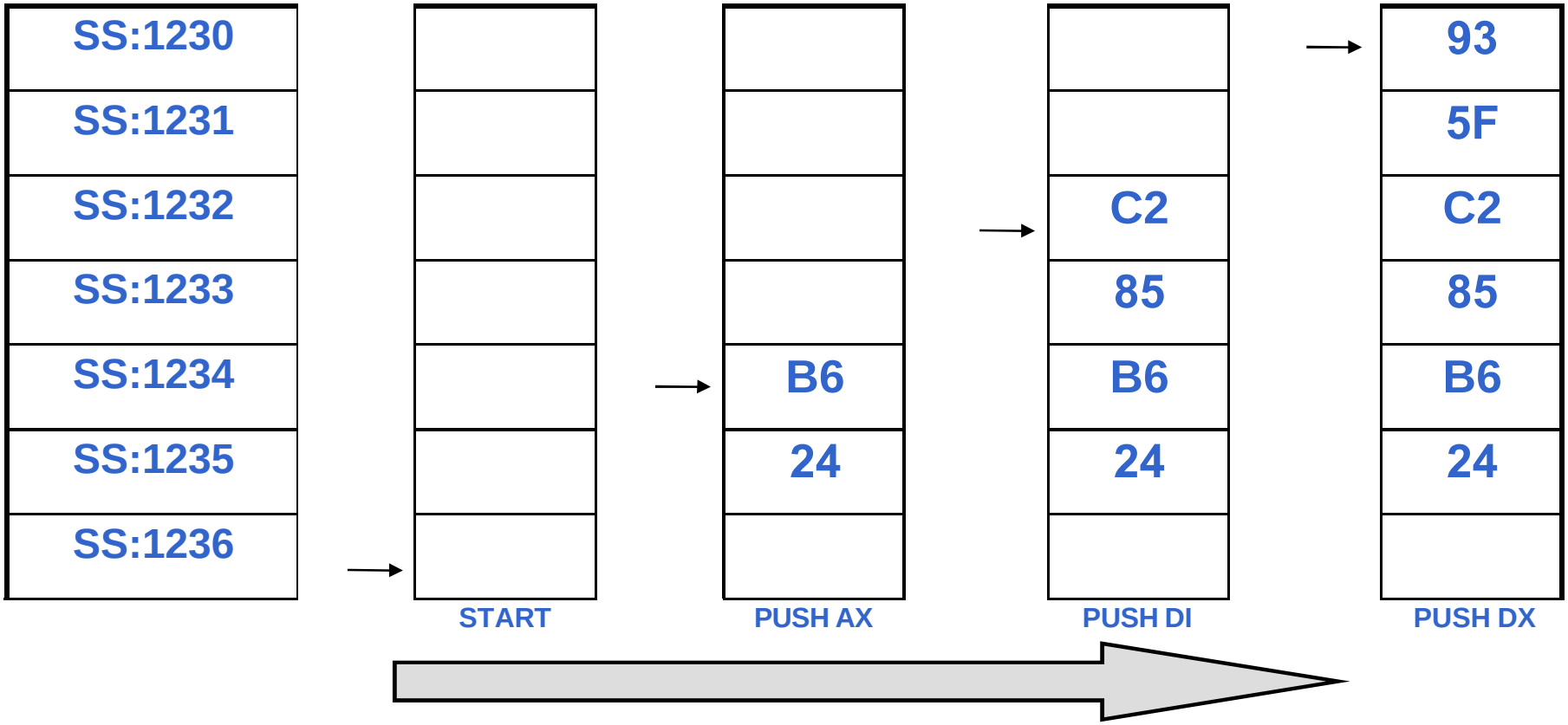
– Assume that SP = 1236, AX = 24B6, DI = 85C2, DX = 5F93.

Show the contents of the stack as each of the following instructions is executed:

PUSH AX

PUSH DI

PUSH DX

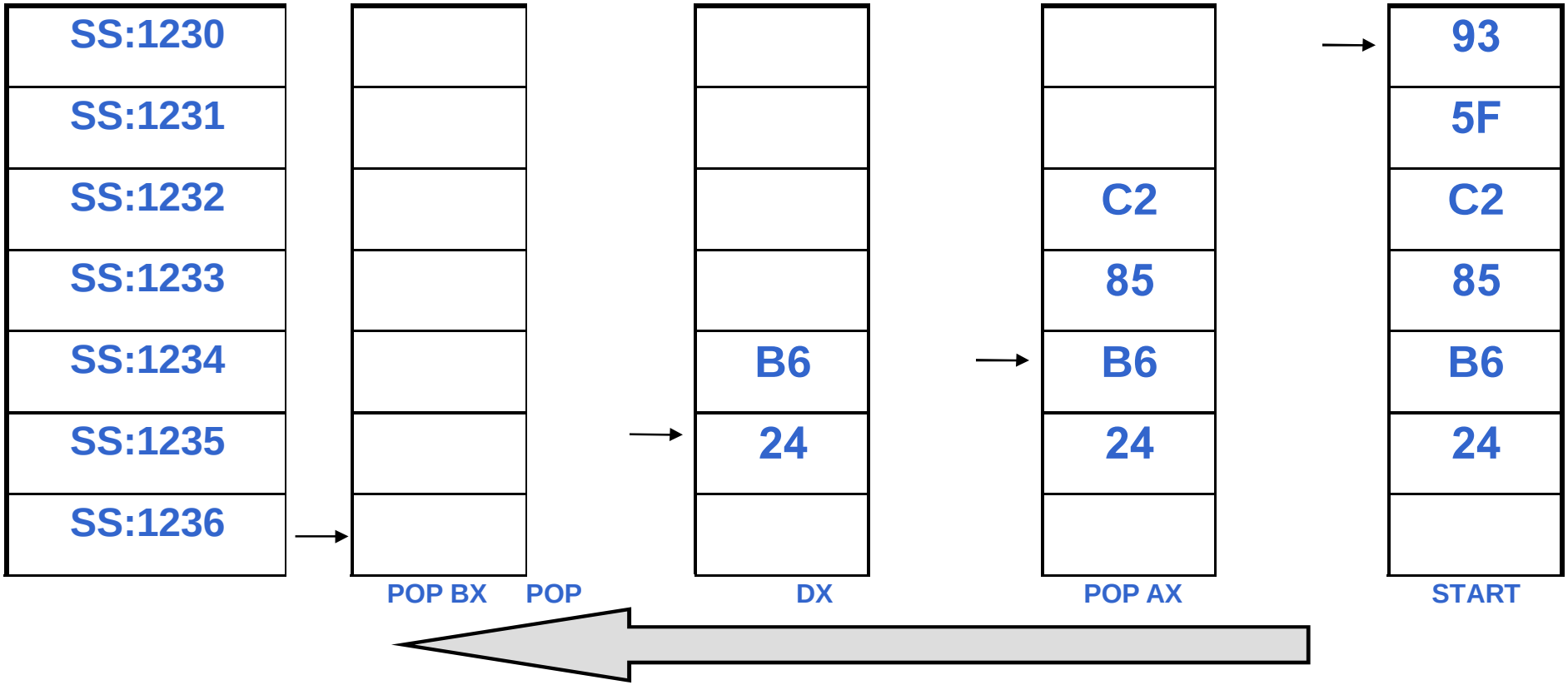


Popping from the Stack

– Popping from the stack, with SP=1230,

show the contents of the stack and registers as each of the following instructions is executed:

POP AX
POP DX
POP BX



Logical vs. Physical Address in the Stack Segment

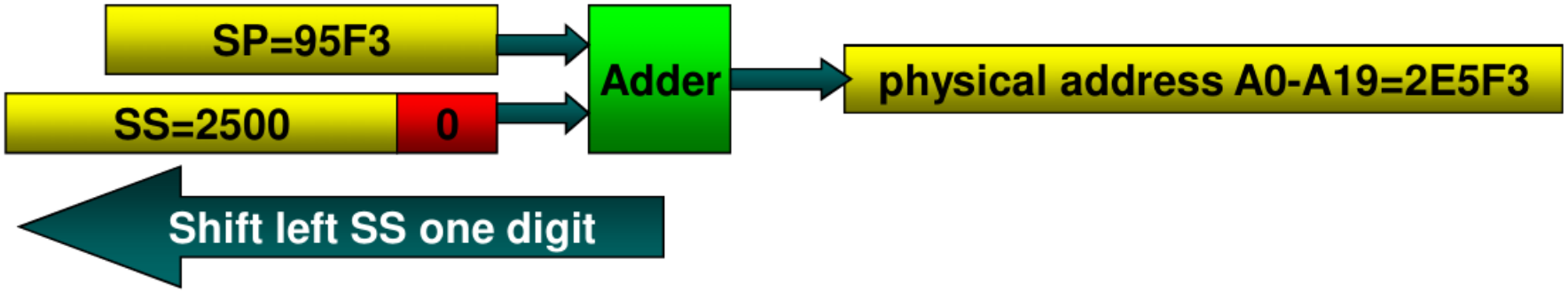
In the stack segment, the SS and SP registers hold the logical address of the current stack location.

The format is SS: SP

Where:

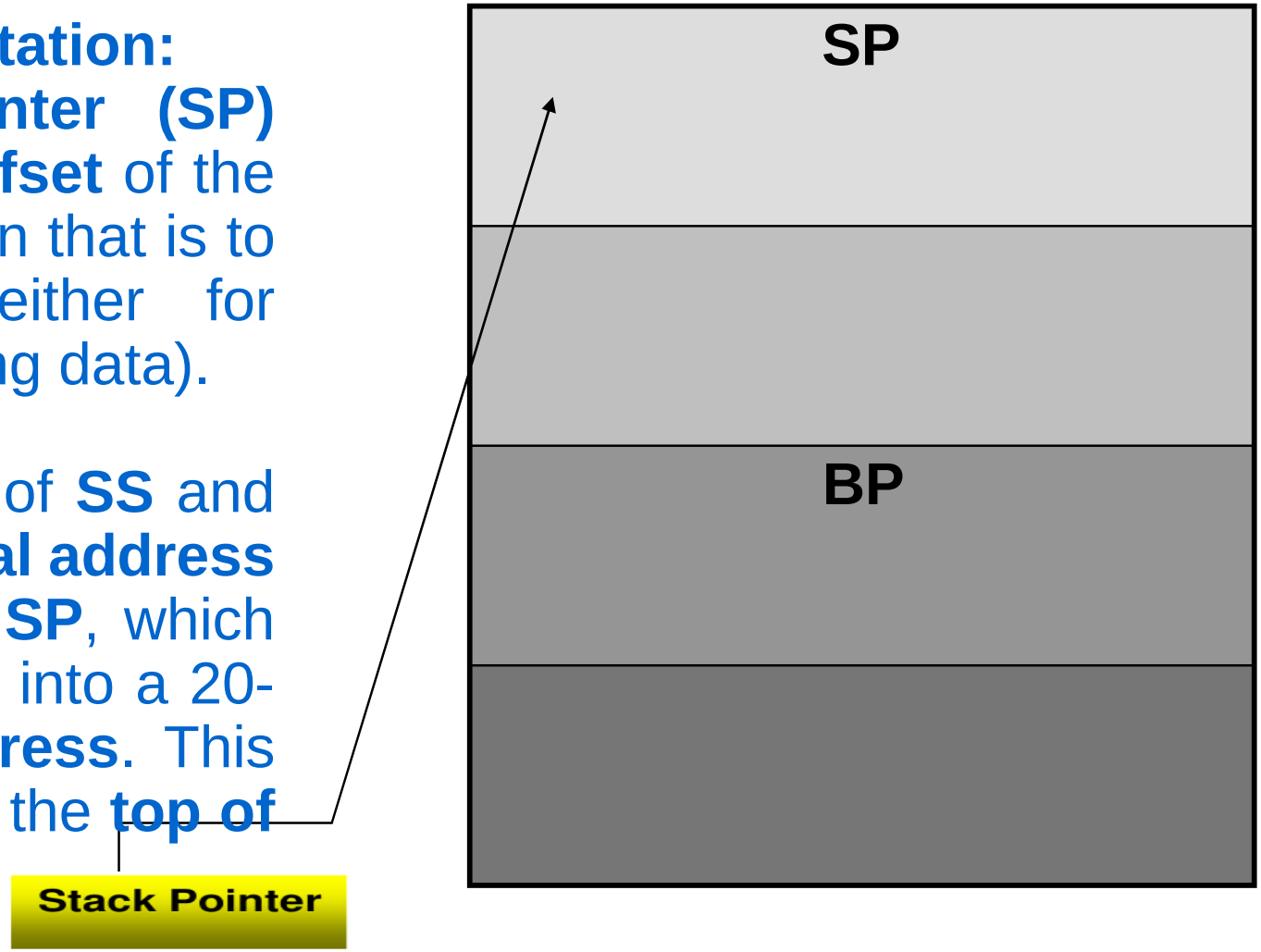
- **SS**: Contains the segment value of the stack.
- SP: Contains the offset value from the beginning of the stack.

Example:



SP – Stack Pointer

- **Offset Representation:**
  The **Stack Pointer (SP)** represents the **offset** of the next stack location that is to be accessed (either for pushing or popping data).
- **(SS:SP):**
  The combination of **SS** and **SP** forms a **logical address** in the format **SS:SP**, which is then converted into a 20-bit **physical address**. This address points to the **top of the stack (TOS)**.



Extra Facts About the Stack

Top of the stack **(TOS)** is the **last stack location occupied**. This is where the most recent data is stored.

The **BP** is another register that can be used as an offset into the stack. It is mainly used to access parameters passed between assembly and high-level Language.

All registers, except the Segment Registers and SP, can be pushed to or popped from the stack.

Data Registers

- **General Purpose Registers:** 8088/ 8086 microprocessor has 4 general purpose 16 bits data registers.

- **Temporary Storage:** These registers are used for **temporary storage** of data during operations.

- **Source and Destination:** Any of the data registers can be utilized as a **source** or **destination** operand in various instructions.

- **8-Bit Usage:** Each 16-bit register can also be accessed as **two separate 8-bit registers**, allowing for more flexible data handling.

Data Registers

AX	
AH	AL
BX	
BH	BL
CX	
CH	CL
DX	
DH	DL

A : Accumulator register

B : Base Register

C: Count Register

D: Data Register

General Purpose Registers

1. **A: Accumulator Registe**r: Primarily used for ***arithmetic and logic operations***. It holds the result of calculations and is often involved in input/output operations.
2. **B: Base Register:** Used to hold the ***based address for memory access***. It assists in addressing data in memory, particularly for indexed addressing modes.
3. **C: Count Register:** Primarily used for ***loop control and counting operations***. It holds the count of iterations for certain instructions, like loops or string operations.
4. **D: Dara Register:** Used to hold data being processed. It can also serve as ***a source or destination*** for data transfer operations.

Data Registers

AX	
AH	AL
BX	
BH	BL
CX	
CH	CL
DX	
DH	DL

Register	Operation
AX	Word operations: Multiply, Divide, Input/Output
AL	Byte operations: Multiply, Divide, Input/Output, Translate, Decimal Arithmetic.
AH	Byte operations: Multiply, Divide
BX	Used for data translation
CX	String operations and loop control
CL	Variable shift and rotate operations
DX	Word operations: Multiply, Divide, and Indirect I/O

Conclusion:

In conclusion, the 8088/8086 microprocessor architecture offers a comprehensive framework for managing data and instructions through a well-defined system of registers and memory segmentation. The distinction between logical and physical addresses enables the processor to efficiently access memory within the constraints of 64 KB segments, allowing a total addressable memory space of 1 MB. Segments such as code, data, stack, and extra, are each assigned specific registers (CS, DS, SS, ES) to manage their base addresses. Additionally, the parallel operation of the Bus Interface Unit (BIU) and Execution Unit (EU) enhances overall performance by simultaneously handling instruction fetching and execution. The role of the stack, managed by the SS and SP registers, is critical for temporary data storage, particularly during function calls and register saves. Understanding these architectural features is essential for optimizing the performance of microprocessor-based systems and writing efficient assembly programs.

References:

1- Mazidi, Muhammad Ali, et al. (2016) The x86 PC Assembly Language, Design, and Interfacing. 5th ed. Upper Saddle River, NJ: Pearson.

- *Mazidi, Muhammad Ali, et al. (2016) 80X86 IBM PC and Compatible Computers: Assembly Language, Design, and Interfacing Volumes I & II. United States ed. Upper Saddle River, NJ: Pearson*.

2- Bery, B.B. (2010) The Intel microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, and Pentium 4: Architecture, Programming, and Interfacing. 8th ed. Upper Saddle River, NJ: Pearson.